11. Attestation Service 구현 분석

개요

AttestationService는 Lighthouse 검증자 클라이언트의 핵심 서비스 중 하나로, 검증자들의 증명(attestation) 생성, 서명, 제출을 담당합니다. 이 서비스는 Ethereum 2.0 합의 프로토콜에서 검증자가 수행해야 하는 가장 중요한 듀티 중 하나인 증명 과정을 자동화하고 관리합니다.

AttestationService는 다음과 같은 주요 기능을 제공합니다:

- 개별 증명(unaggregated attestation) 생성 및 제출
- 집계 증명(aggregated attestation) 생성 및 제출
- 슬래싱 보호 메커니즘 통합
- 다중 검증자 지원 및 최적화

아키텍처 및 구조

AttestationService 구조체

```
// validator_client/validator_services/src/attestation_service.rs

pub struct AttestationService<S, T> {

inner: Arc<Inner<S, T>>,

}


pub struct Inner<S, T> {

duties_service: Arc<DutiesService<S, T>>,
```

```
validator_store: Arc<S>,
slot_clock: T,
beacon_nodes: Arc<BeaconNodeFallback<T>>,
executor: TaskExecutor,
chain_spec: Arc<ChainSpec>,
disable: bool,
}
```

AttestationService는 다음과 같은 핵심 컴포넌트들과 상호작용합니다:

1. **DutiesService**: 검증자 듀티 정보 제공
2. **ValidatorStore**: 키 관리 및 서명 기능
3. **BeaconNodeFallback**: 비콘 노드와의 통신
4. **SlotClock**: 시간 동기화
5. **TaskExecutor**: 비동기 작업 실행

Builder 패턴 구현

AttestationService는 Builder 패턴을 사용하여 초기화됩니다:

```
impl<S: ValidatorStore + 'static, T: SlotClock + 'static> AttestationServiceBuil
pub fn new() -> Self { /* ... */ }
pub fn duties_service(mut self, service: Arc<DutiesService<S, T>>) -> Self {
pub fn validator_store(mut self, store: Arc<S>) -> Self { /* ... */ }
pub fn slot_clock(mut self, slot_clock: T) -> Self { /* ... */ }
pub fn beacon_nodes(mut self, beacon_nodes: Arc<BeaconNodeFallback<T>>) -> Se
```

```
pub fn executor(mut self, executor: TaskExecutor) -> Self { /* ... */ }

pub fn chain_spec(mut self, chain_spec: Arc<ChainSpec>) -> Self { /* ... */ }

pub fn build(self) -> Result<AttestationService<S, T>, String> { /* ... */ }

}
```

서비스 시작 및 스케줄링

start_update_service() 메서드

AttestationService는 `start_update_service()` 메서드를 통해 주기적인 증명 생성 프로세스를 시작합니다:

```
pub fn start_update_service(self, spec: &ChainSpec) -> Result<(), String> {

if self.disable {

info!("Attestation service disabled");

return Ok(());

}


let slot_duration = Duration::from_secs(spec.seconds_per_slot);

let duration_to_next_slot = self

.slot_clock

.duration_to_next_slot()

.ok_or("Unable to determine duration to next slot")?;
```

```
let executor = self.executor.clone();

let interval_fut = async move {
loop {
if let Some(duration_to_next_slot) = self.slot_clock.duration_to_next_slot() {
// 슬롯의 1/3 지점에서 증명 작업 시작
sleep(duration_to_next_slot + slot_duration / 3).await;

if let Err(e) = self.spawn_attestation_tasks(slot_duration) {
crit!(error = e, "Failed to spawn attestation tasks")
} else {
trace!("Spawned attestation tasks");
}
} else {
error!("Failed to read slot clock");
sleep(slot_duration).await;
continue;
}
}
};

executor.spawn(interval_fut, "attestation_service");
Ok(())
```

```
}
```

핵심 특징:

- 각 슬롯의 1/3 지점에서 증명 작업을 시작
- 슬롯 클록 동기화를 통한 정확한 타이밍 제어
- 에러 발생 시 로깅 및 복구 메커니즘

spawn_attestation_tasks() 메서드

이 메서드는 현재 슬롯에 대한 모든 증명 작업을 스폰합니다:

```
fn spawn_attestation_tasks(&self, slot_duration: Duration) -> Result<(), String

let slot = self.slot_clock.now().ok_or("Failed to read slot clock")?;

let duration_to_next_slot = self

.slot_clock

.duration_to_next_slot()

.ok_or("Unable to determine duration to next slot")?;


// 집계 증명은 슬롯의 2/3 지점에서 생성

let aggregate_production_instant = Instant::now()

+ duration_to_next_slot

.checked_sub(slot_duration / 3)

.unwrap_or_else(|| Duration::from_secs(0));


// 위원회 인덱스별로 듀티 그룹화
```

```
let duties_by_committee_index: HashMap<CommitteeIndex, Vec<DutyAndProof>> = sel
.duties_service
.attesters(slot)
.into_iter()
.fold(HashMap::new(), |mut map, duty_and_proof| {
map.entry(duty_and_proof.duty.committee_index)
.or_default()
.push(duty_and_proof);
map
});


// 각 위원회별로 별도 작업 스폰
duties_by_committee_index
.into_iter()
.for_each(|(committee_index, validator_duties)| {
self.inner.executor.spawn_ignoring_error(
self.clone().publish_attestations_and_aggregates(
slot,
committee_index,
validator_duties,
aggregate_production_instant,
),
"attestation publish",
```

```
);
});

// 슬래싱 보호 데이터베이스 정리 작업 스케줄링
self.spawn_slashing_protection_pruning_task(slot, aggregate_production_instant);

Ok(())
}
```

최적화 특징:

- 위원회 인덱스별로 듀티를 그룹화하여 효율성 향상
- 각 위원회별로 독립적인 비동기 작업 실행
- 집계 증명 생성을 위한 정확한 타이밍 계산

핵심 증명 생성 프로세스

publish_attestations_and_aggregates() 메서드

이 메서드는 증명 생성의 전체 워크플로우를 관리합니다:

```
async fn publish_attestations_and_aggregates(
self,
slot: Slot,
committee_index: CommitteeIndex,
validator_duties: Vec<DutyAndProof>,
```

```
aggregate_production_instant: Instant,

) -> Result<(), ()> {

let  attestations_timer = validator_metrics::start_timer_vec(

&validator_metrics::ATTESTATION_SERVICE_TIMES,

&[validator_metrics::ATTESTATIONS],

);



if  validator_duties.is_empty() {

return Ok(());

}



// Step 1: 개별 증명 생성 및 제출

let  attestation_opt = self

.produce_and_publish_attestations(slot, committee_index, &validator_duties)

.await

.map_err(move |e| {

crit!(

error = format!("{:?}", e),

committee_index,

slot = slot.as_u64(),

"Error during attestation routine"

)

})?;
```

```
drop(attestations_timer);


// Step 2: 집계 증명 생성 및 제출
if  let Some(attestation_data) = attestation_opt {
// 집계 생성 시점까지 대기 (슬롯의 2/3 지점)
sleep_until(aggregate_production_instant).await;


let  _aggregates_timer = validator_metrics::start_timer_vec(
&validator_metrics::ATTESTATION_SERVICE_TIMES,
&[validator_metrics::AGGREGATES],
);


self.produce_and_publish_aggregates(
&attestation_data,
committee_index,
&validator_duties,
)
.await
.map_err(move |e| {
crit!(
error = format!("{:?}", e),
committee_index,
```

```
slot = slot.as_u64(),
"Error during attestation routine"
)
})?;
}

Ok(())
}
```

2단계 프로세스:

1. **개별 증명 단계**: 각 검증자의 개별 증명 생성 및 제출
2. **집계 증명 단계**: 집계자로 선정된 검증자들의 집계 증명 생성

produce_and_publish_attestations() 상세 분석

이 메서드는 AttestationService의 핵심 기능으로, 개별 증명의 전체 생명주기를 관리합니다:

1. 증명 데이터 다운로드

```
let attestation_data = self
.beacon_nodes
.first_success(|beacon_node| async move {
let _timer = validator_metrics::start_timer_vec(
&validator_metrics::ATTESTATION_SERVICE_TIMES,
&[validator_metrics::ATTESTATIONS_HTTP_GET],
```

```
);
beacon_node
.get_validator_attestation_data(slot, committee_index)
.await
.map_err(|e| format!("Failed to produce attestation data: {:?}", e))
.map(|result| result.data)
})
.await
.map_err(|e| e.to_string())?;
```

특징:

- BeaconNodeFallback을 통한 고가용성 보장
- 메트릭 수집을 통한 성능 모니터링
- 에러 처리 및 로깅

2. 병렬 서명 처리

```
let signing_futures = validator_duties.iter().map(|duty_and_proof| async move
let duty = &duty_and_proof.duty;
let attestation_data = attestation_data_ref;

// 듀티와 증명 데이터 일치성 검증
if !duty.match_attestation_data::<S::E>(attestation_data, &self.chain_spec) {
crit!(
```

```
validator = ?duty.pubkey,
duty_slot = %duty.slot,
attestation_slot = %attestation_data.slot,
duty_index = duty.committee_index,
attestation_index = attestation_data.index,
"Inconsistent validator duties during signing"
);
return None;
}


// 빈 증명 객체 생성
let mut attestation = match Attestation::empty_for_signing(
duty.committee_index,
duty.committee_length as usize,
attestation_data.slot,
attestation_data.beacon_block_root,
attestation_data.source,
attestation_data.target,
&self.chain_spec,
) {
Ok(attestation) => attestation,
Err(err) => {
crit!(
```

```
validator = ?duty.pubkey,
?duty,
?err,
"Invalid validator duties during signing"
);
return None;
}
};


// ValidatorStore를 통한 서명
match  self
.validator_store
.sign_attestation(
duty.pubkey,
duty.validator_committee_index as usize,
&mut  attestation,
current_epoch,
)
.await
{
Ok(()) => Some((attestation, duty.validator_index)),
Err(ValidatorStoreError::UnknownPubkey(pubkey)) => {
warn!(
```

```
info = "a validator may have recently been removed from this VC",

pubkey = ?pubkey,

validator = ?duty.pubkey,

committee_index = committee_index,

slot = slot.as_u64(),

"Missing pubkey for attestation"

);

None

}

Err(e) => {

crit!(

error = ?e,

validator = ?duty.pubkey,

committee_index,

slot = slot.as_u64(),

"Failed to sign attestation"

);

None

}

}

});


// 모든 서명 작업을 병렬로 실행
```

```
let (ref attestations, ref validator_indices): (Vec<_>, Vec<_>) = join_all(sig
.await
.into_iter()
.flatten()
.unzip();
```

핵심 특징:

- **병렬 처리**: `join_all()` 을 사용하여 모든 검증자의 서명을 동시에 처리
- **데이터 검증**: 듀티와 증명 데이터의 일치성 검증
- **에러 처리**: 각 검증자별 독립적인 에러 처리
- **슬래싱 보호**: ValidatorStore를 통한 안전한 서명

3. 증명 제출

```
match self
.beacon_nodes
.request(ApiTopic::Attestations, |beacon_node| async move {
let _timer = validator_metrics::start_timer_vec(
&validator_metrics::ATTESTATION_SERVICE_TIMES,
&[validator_metrics::ATTESTATIONS_HTTP_POST],
);

let single_attestations = attestations
```

```
.iter()

.zip(validator_indices)

.filter_map(|(a, i)| {

match  a.to_single_attestation_with_attester_index(*i) {

Ok(a) => Some(a),

Err(e) => {

error!(

error = ?e,

committee_index = attestation_data.index,

slot = slot.as_u64(),

"type" = "unaggregated",

"Unable to convert to SingleAttestation"

);

None

}

}

})

.collect::<Vec<_>>();



beacon_node

.post_beacon_pool_attestations_v2::<S::E>(single_attestations, fork_name)

.await

})
```

```
.await
{
Ok(()) => info!(
count = attestations.len(),
validator_indices = ?validator_indices,
head_block = ?attestation_data.beacon_block_root,
committee_index = attestation_data.index,
slot = attestation_data.slot.as_u64(),
"type" = "unaggregated",
"Successfully published attestations"
),
Err(e) => error!(
error = %e,
committee_index = attestation_data.index,
slot = slot.as_u64(),
"type" = "unaggregated",
"Unable to publish attestations"
),
}
```

특징:

- **포크 호환성**: Electra 포크 지원을 위한 버전별 API 호출
- **배치 제출**: 여러 증명을 한 번에 제출하여 효율성 향상

- **상세한 로깅**: 성공/실패 상황에 대한 구체적인 정보 제공

집계 증명 처리

produce_and_publish_aggregates() 메서드

집계 증명은 네트워크 효율성을 위해 여러 개별 증명을 하나로 합치는 과정입니다:

```
async fn produce_and_publish_aggregates(
&self,
attestation_data: &AttestationData,
committee_index: CommitteeIndex,
validator_duties: &[DutyAndProof],
) -> Result<(), String> {
// 집계자가 없으면 조기 종료
if !validator_duties
.iter()
.any(|duty_and_proof| duty_and_proof.selection_proof.is_some())
{
return Ok(());
}

let fork_name = self
.chain_spec
.fork_name_at_slot::<S::E>(attestation_data.slot);
```

```
// 집계된 증명 다운로드
let  aggregated_attestation = &self
.beacon_nodes
.first_success(|beacon_node| async  move {
let  _timer = validator_metrics::start_timer_vec(
&validator_metrics::ATTESTATION_SERVICE_TIMES,
&[validator_metrics::AGGREGATES_HTTP_GET],
);
if  fork_name.electra_enabled() {
beacon_node
.get_validator_aggregate_attestation_v2(
attestation_data.slot,
attestation_data.tree_hash_root(),
committee_index,
)
.await
.map_err(|e| {
format!("Failed to produce an aggregate attestation: {:?}", e)
})?
.ok_or_else(|| format!("No aggregate available for {:?}", attestation_data))
.map(|result| result.into_data())
} else {
```

```
beacon_node
.get_validator_aggregate_attestation_v1(
attestation_data.slot,
attestation_data.tree_hash_root(),
)
.await
.map_err(|e| {
format!("Failed to produce an aggregate attestation: {:?}", e)
})?
.ok_or_else(|| format!("No aggregate available for {:?}", attestation_data))
.map(|result| result.data)
}
})
.await
.map_err(|e| e.to_string())?;


// 집계자들의 서명된 집계 증명 생성
let  signing_futures = validator_duties.iter().map(|duty_and_proof| async  move
let  duty = &duty_and_proof.duty;
let  selection_proof = duty_and_proof.selection_proof.as_ref()?;


if !duty.match_attestation_data::<S::E>(attestation_data, &self.chain_spec) {
crit!("Inconsistent validator duties during signing");
```

```
return None;
}


match  self
.validator_store
.produce_signed_aggregate_and_proof(
duty.pubkey,
duty.validator_index,
aggregated_attestation.clone(),
selection_proof.clone(),
)
.await
{
Ok(aggregate) => Some(aggregate),
Err(ValidatorStoreError::UnknownPubkey(pubkey)) => {
debug!(?pubkey, "Missing pubkey for aggregate");
None
}
Err(e) => {
crit!(
error = ?e,
pubkey = ?duty.pubkey,
"Failed to sign aggregate"
```

```
);
None
}
}
});

let signed_aggregate_and_proofs = join_all(signing_futures)
.await
.into_iter()
.flatten()
.collect::<Vec<_>>();

// 집계 증명 제출
if !signed_aggregate_and_proofs.is_empty() {
let signed_aggregate_and_proofs_slice = signed_aggregate_and_proofs.as_slice();
match self
.beacon_nodes
.first_success(|beacon_node| async move {
let _timer = validator_metrics::start_timer_vec(
&validator_metrics::ATTESTATION_SERVICE_TIMES,
&[validator_metrics::AGGREGATES_HTTP_POST],
);
if fork_name.electra_enabled() {
```

```
beacon_node
.post_validator_aggregate_and_proof_v2(
signed_aggregate_and_proofs_slice,
fork_name,
)
.await
} else {
beacon_node
.post_validator_aggregate_and_proof_v1(
signed_aggregate_and_proofs_slice,
)
.await
}
})
.await
{
Ok(()) => {
for signed_aggregate_and_proof in signed_aggregate_and_proofs {
let attestation = signed_aggregate_and_proof.message().aggregate();
info!(
aggregator = signed_aggregate_and_proof.message().aggregator_index(),
signatures = attestation.num_set_aggregation_bits(),
head_block = format!("{:?}", attestation.data().beacon_block_root),
```

```
committee_index = attestation.committee_index(),
slot = attestation.data().slot.as_u64(),
"type" = "aggregated",
"Successfully published attestation"
);
}
}
Err(e) => {
for  signed_aggregate_and_proof  in  signed_aggregate_and_proofs {
let  attestation = &signed_aggregate_and_proof.message().aggregate();
crit!(
error = %e,
aggregator = signed_aggregate_and_proof.message().aggregator_index(),
committee_index = attestation.committee_index(),
slot = attestation.data().slot.as_u64(),
"type" = "aggregated",
"Failed to publish attestation"
);
}
}
}
}
```

```
Ok(())
}
```

집계 증명의 특징:

- **선택적 처리**: selection_proof가 있는 검증자만 집계 작업 수행
- **포크 호환성**: Electra 포크 이전/이후 버전별 API 지원
- **효율적 네트워크 사용**: 여러 개별 증명을 하나로 집계하여 대역폭 절약

슬래싱 보호 통합

AttestationService는 슬래싱 보호를 위한 데이터베이스 정리 작업도 관리합니다:

```
fn spawn_slashing_protection_pruning_task(&self, slot: Slot, pruning_instant: I
let attestation_service = self.clone();
let executor = self.inner.executor.clone();
let current_epoch = slot.epoch(S::E::slots_per_epoch());

// pruning_instant에서 정리 작업 시작
self.inner.executor.spawn(
async move {
sleep_until(pruning_instant).await;

executor.spawn_blocking(
move || {
```

```
attestation_service
.validator_store
.prune_slashing_protection_db(current_epoch, false)
},
"slashing_protection_pruning",
)
},
"slashing_protection_pre_pruning",
);
}
```

특징:

- **타이밍 최적화**: 집계 증명 생성과 동일한 시점에 실행
- **블로킹 작업 분리**: 별도 스레드에서 데이터베이스 정리 수행
- **성능 유지**: 정기적인 정리를 통한 데이터베이스 성능 보장

데이터 구조 및 타입

DutyAndProof 구조체

```
#[derive(Clone)]
pub struct DutyAndProof {
pub duty: AttesterData,
/// 집계자인 경우에만 설정되는 선택 증명
```

```
pub selection_proof: Option<SelectionProof>,
/// 구독 메시지를 보낼 슬롯들을 추적
pub subscription_slots: Arc<SubscriptionSlots>,
}
```

AttestationData 구조체

```
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize, Hash, Encode, Deco
pub struct AttestationData {
pub slot: Slot,
pub index: u64,
// LMD GHOST 투표
pub beacon_block_root: Hash256,
// FFG 투표
pub source: Checkpoint,
pub target: Checkpoint,
}
```

Attestation 구조체

```
pub struct Attestation<E: EthSpec> {
// Base 포크: MaxValidatorsPerCommittee 크기
// Electra 포크: MaxValidatorsPerSlot 크기
```

```
pub aggregation_bits: BitList<E::MaxValidatorsPerCommittee>,
pub data: AttestationData,
pub signature: AggregateSignature,
// Electra 포크에서만 사용
pub committee_bits: BitVector<E::MaxCommitteesPerSlot>,
}
```

성능 최적화 및 메트릭

AttestationService는 다양한 성능 최적화 기법을 사용합니다:

1. 병렬 처리

- 모든 검증자의 서명을 `join_all()` 로 병렬 처리
- 위원회별 독립적인 비동기 작업 실행

2. 배치 처리

- 여러 증명을 한 번의 HTTP 요청으로 제출
- 네트워크 오버헤드 최소화

3. 메트릭 수집

```
let attestations_timer = validator_metrics::start_timer_vec(
&validator_metrics::ATTESTATION_SERVICE_TIMES,
&[validator_metrics::ATTESTATIONS],
```

```
);
```

4. 리소스 관리

- Arc를 통한 효율적인 메모리 공유
- 적절한 타이밍에서의 슬래싱 보호 DB 정리

에러 처리 및 복구

AttestationService는 견고한 에러 처리 메커니즘을 제공합니다:

1. 검증자별 독립적 처리

- 한 검증자의 실패가 다른 검증자에게 영향을 주지 않음
- 각 검증자별 상세한 에러 로깅

2. 네트워크 장애 대응

- BeaconNodeFallback을 통한 자동 장애 조치
- 재시도 메커니즘 내장

3. 슬래싱 보호

- ValidatorStore를 통한 안전한 서명
- 중복 서명 방지 메커니즘

실제 사용 예시

AttestationService의 전체 워크플로우는 다음과 같습니다:

1. **서비스 시작**: `start_update_service()` 호출

2. **슬롯 동기화**: 각 슬롯의 1/3 지점에서 작업 시작

3. **듀티 수집**: DutiesService에서 현재 슬롯의 증명 듀티 조회

4. **위원회별 그룹화**: 효율적인 처리를 위한 듀티 그룹화

5. **개별 증명 생성**:

- 비콘 노드에서 AttestationData 다운로드
- 각 검증자별 병렬 서명 처리
- 배치로 증명 제출

6. **집계 증명 생성**:

- 슬롯의 2/3 지점까지 대기
- 집계된 증명 다운로드
- 집계자들의 SignedAggregateAndProof 생성 및 제출

7. **정리 작업**: 슬래싱 보호 데이터베이스 정리

이러한 체계적인 프로세스를 통해 AttestationService는 Ethereum 2.0 네트워크에서 검증자의 핵심 역할인 증명 생성을 안전하고 효율적으로 수행합니다.